

بسمه تعالی

پروژه درس برنامه نویسی پیشرفته

دانشکده فنی انقلاب اسلامی

دانشگاه ملی مهارت

استاد درس: دکتر اردستانی

نام و نام خانوادگی دانشجو: شایان عباسی

شماره دانشجویی: 04121040768022

نکته: در تمام این پروژه ها a آخرین رقم غیر صفر شماره دانشجویی شماست.

پروژه 3: شبیه‌سازی مخزن آب و کنترل سطح

موضوع کنترل

دانشجو یک مخزن 100 را مدل کند.

ورودی

دبی ورودی

خروجی

ارتفاع آب

امکانات

- رسم نمودار سطح
- کنترل On-Off
- کنترل PID ساده

کلاس‌ها

Tank

Pump

Sensor

Controller

Simulation

نمره ویژه

مقایسه کنترلرها

1. کتابخانه ها

```
import tkinter as tk
```

در این خط، کتابخانه `tkinter` را وارد کردم و نام مستعار `tk` به آن دادم. از این کتابخانه برای ساخت پنجره اصلی برنامه و تمام عناصر گرافیکی مانند `Canvas`، دکمه‌ها، `Scale` و `Radiobutton` استفاده کردیم.

```
from tkinter import ttk
```

از این خط برای وارد کردن ماژول `ttk` از `tkinter` استفاده کردم. `ttk` امکانات پیشرفته‌تری مانند `Radiobutton` با ظاهر بهتر فراهم می‌کند.

```
import matplotlib.pyplot as plt
```

کتابخانه `matplotlib` را برای رسم نمودار سطح آب در طول زمان وارد کردم. این کتابخانه امکان رسم نمودارهای حرفه‌ای را فراهم می‌کند.

```
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
```

این کلاس امکان جاسازی نمودار `matplotlib` داخل پنجره `tkinter` را فراهم می‌کند. با استفاده از آن نمودار مستقیماً در کنار عناصر دیگر UI نمایش داده می‌شود.

۲. کلاس Tank (مدل مخزن)

```
class Tank:
    def __init__(self, max_level=100):
        self.max = max_level
        self.level = 20
```

این کلاس را برای مدل‌سازی مخزن آب طراحی کردم. حداکثر سطح آب ۱۰۰ واحد است و سطح اولیه روی ۲۰ تنظیم شده تا شبیه‌سازی از یک حالت پایین شروع کند و رفتار کنترلر کاملاً مشخص باشد.

متد update به روزرسانی سطح آب

```
def update(self, qin, dt):  
    qout = 0.05 * self.level  
    self.level = self.level + (qin - qout) * dt  
    if self.level < 0:  
        self.level = 0  
    if self.level > self.max:  
        self.level = self.max
```

این متد در هر گام زمانی فراخوانی می‌شود. جریان خروجی $qout$ به صورت خطی به سطح آب وابسته است (مدل ساده تخلیه). سطح آب با معادله تفاضلی به روز می‌شود و در محدوده ۰ تا max نگه داشته می‌شود تا از خطاهای فیزیکی جلوگیری شود.

۳. کلاس Pump (مدل پمپ)

```
class Pump:  
    def __init__(self, max_flow=10):  
        self.max = max_flow  
        self.u = 0
```

این کلاس را برای مدل‌سازی پمپ آب ساختم. حداکثر دبی پمپ ۱۰ واحد است و u سیگنال کنترلی بین ۰ تا ۱ را نگه می‌دارد که توسط کنترلر تنظیم می‌شود.

متد flow محاسبه دبی خروجی

```
def flow(self):  
    return self.u * self.max
```

این متد دبی واقعی پمپ را محاسبه می‌کند. با ضرب سیگنال کنترلی u در حداکثر دبی، جریان ورودی به مخزن به دست می‌آید. وقتی u برابر ۱ باشد پمپ با حداکثر توان کار می‌کند.

۴. کلاس Sensor (حسگر سطح)

```
class Sensor:
    def __init__(self, tank):
        self.tank = tank
```

این کلاس را برای شبیه‌سازی حسگر سطح آب طراحی کردم. ارجاع به مخزن را دریافت می‌کند تا بتواند سطح آب را بخواند. این جداسازی کمک می‌کند در آینده نویز یا خطای اندازمگیری به راحتی اضافه شود.

متد read خواندن سطح آب

```
def read(self):
    return self.tank.level
```

این متد سطح فعلی آب را از مخزن می‌خواند و برمی‌گرداند. در loop اصلی برنامه، هر بار قبل از محاسبه کنترلر این متد فراخوانی می‌شود تا مقدار واقعی سطح آب دریافت شود.

۵. کلاس OnOffController (کنترلر دو حالت)

```
class OnOffController:
    def __init__(self, sp, h=2):
        self.sp = sp
        self.h = h
        self.state = 0
```

این کلاس را برای پیاده‌سازی کنترلر ساده دو حالت طراحی کردم. sp نقطه تنظیم و h هیسترزیس است که از نوسان سریع پمپ جلوگیری می‌کند. state وضعیت فعلی پمپ را نگه می‌دارد.

متد compute محاسبه خروجی On-Off

```
def compute(self, y):
    if y < self.sp - self.h:
        self.state = 1
    elif y > self.sp + self.h:
        self.state = 0
```

```
return self.state
```

این متد سطح فعلی را می‌گیرد و تصمیم می‌گیرد پمپ روشن باشد یا خاموش. اگر سطح پایین‌تر از sp منهای هیستریزیس باشد پمپ روشن می‌شود و اگر بالاتر از sp به علاوه هیستریزیس برود خاموش می‌شود. هیستریزیس از تغییر مداوم حالت جلوگیری می‌کند.

۶. کلاس PIDController (کنترلر PID)

```
class PIDController:
```

```
    def __init__(self, sp, kp, ki, kd):  
        self.sp = sp  
        self.kp = kp  
        self.ki = ki  
        self.kd = kd  
        self.i = 0  
        self.prev = 0
```

این کلاس را برای پیاده‌سازی کنترلر PID طراحی کردم. kp ضریب تناسبی، ki ضریب انتگرالی و kd ضریب مشتقی است. متغیر i انتگرال خطا را انباشته می‌کند و $prev$ خطای قبلی را برای محاسبه مشتق نگه می‌دارد.

متد compute محاسبه خروجی PID

```
def compute(self, y, dt):  
    e = self.sp - y  
    self.i += e * dt  
    d = (e - self.prev) / dt  
    self.prev = e  
    u = self.kp * e + self.ki * self.i + self.kd * d  
    if u < 0: u = 0  
    if u > 1: u = 1  
    return u
```

این متد سه جزء PID را محاسبه می‌کند. خطا e برابر نقطه تنظیم منهای مقدار واقعی است. جزء انتگرالی خطاهای گذشته را جمع می‌کند و جزء مشتقی سرعت تغییر خطا را اندازه می‌گیرد. خروجی نهایی بین ۰ و ۱ محدود می‌شود تا از محدوده کار پمپ خارج نشود.

۷. کلاس App (رابط گرافیکی)

```
class App:
    def __init__(self, root):
        self.root = root
        self.root.title("Tank Level Control")
        self.tank = Tank()
        self.pump = Pump()
        self.sensor = Sensor(self.tank)
        self.running = False
        self.dt = 0.1
        self.t = 0
        self.x = []
        self.y = []
```

این کلاس را برای مدیریت کامل رابط گرافیکی طراحی کردم. مخزن، پمپ و حسگر در سازنده ساخته می‌شوند. **running** وضعیت اجرای شبیه‌سازی، **dt** گام زمانی و **x** و **y** داده‌های نمودار را نگه می‌دارند.

متدهای start / stop / reset کنترل شبیه‌سازی

```
def start(self):
    self.running = True
def stop(self):
    self.running = False
def reset(self):
    self.tank = Tank()
    self.pump = Pump()
    self.sensor = Sensor(self.tank)
    self.t = 0
    self.x = []
    self.y = []
```

این سه متد کنترل اجرای شبیه‌سازی را مدیریت می‌کنند. **start** شبیه‌سازی را فعال می‌کند، **stop** آن را متوقف می‌کند و **reset** همه اجزا را از نو می‌سازد و داده‌های نمودار را پاک می‌کند تا شبیه‌سازی از ابتدا شروع شود.

متد loop حلقه اصلی شبیه‌سازی

```
def loop(self):
    if self.running:
        level = self.sensor.read()
        if self.mode.get() == "PID":
            self.pump.u = self.pid.compute(level, self.dt)
        else:
            self.pump.u = self.on.compute(level)
        self.tank.update(self.pump.flow(), self.dt)
        self.t += self.dt
        self.x.append(self.t)
        self.y.append(self.tank.level)
        self.line.set_data(self.x, self.y)
        self.figcanvas.draw()
    self.root.after(100, self.loop)
```

این متد مهم‌ترین بخش برنامه است و هر ۱۰۰ میلی‌ثانیه اجرا می‌شود. ابتدا سطح آب از حسگر خوانده می‌شود، سپس بر اساس حالت انتخابی کنترلر PID یا On-Off سیگنال کنترلی محاسبه می‌شود. مخزن به‌روزرسانی شده و داده‌ها به لیست‌های نمودار اضافه می‌شوند. در نهایت نمودار و تصویر مخزن بازرسم می‌شوند.

متد draw canvas رسم تصویر مخزن

```
self.canvas.delete("all")
self.canvas.create_rectangle(50,20,110,280)
h = (self.tank.level/100)*260
self.canvas.create_rectangle(51,280-h,109,279,fill="deepskyblue")
self.canvas.create_text(80,300,text="Level: %.1f" % self.tank.level)
```

این بخش از loop تصویر گرافیکی مخزن را رسم می‌کند. یک مستطیل خالی به عنوان بدنه مخزن رسم می‌شود و درون آن یک مستطیل آبی رنگ ارتفاع آب را نشان می‌دهد. ارتفاع به نسبت سطح واقعی محاسبه می‌شود تا تصویر با مقدار واقعی همخوانی داشته باشد.

۸. نقطه ورودی برنامه

```
root = tk.Tk()
app = App(root)
root.mainloop()
```

این بلوک نقطه شروع برنامه است. یک پنجره `tkinter` ساخته می‌شود، آن را به `App` می‌دهیم و با `mainloop()` برنامه را در حالت اجرا نگه می‌داریم تا کاربر پنجره را ببندد.